MICRO-PROYECTO 1-SISTEMAS DIGITALES 2: CONTRUYE UN MICROPROCESADOR RISC-V

GUÍA 3

Hasta ahora hemos implementado un procesador RISC-V que realiza la mayoría de las instrucciones aritmético-lógicas. Sí recuerda el ciclo de ejecución de un microprocesador, éste debe capturar la instrucción apuntada por el PC, decodificarla y luego ejecutarla. Aunque parte de la decodificación y ejecución están implementados en la Unidad de Control (Control Unit), hace falta incluir la lógica del PC y todas las instrucciones que lo modifican según la condición (instrucciones tipo B, Branch, e instrucciones tipo J, Jump).

PARA ESTUDIO INDEPENDIENTE FUERA DE CLASE

6 – Instrucciones de salto

El microprocesador no estaría completo si no tuviera instrucciones para implementar directivas de alto nivel como “if”, ciclos “for”, ciclos “while” y “do…while”. En estos casos se usan instrucciones de salto condicional e incondicional.

En los siguientes ejemplos se muestra cómo usar las instrucciones de salto para implementar un “if … else”, y los ciclos “do…while” y “while”, note que en estos casos se usan etiquetas como argumentos inmediatos para los saltos. Los ensambladores resuelven las etiquetas asignándole el valor numérico correcto cuando generan los códigos de máquina (*machine codes*).

Ejemplo 1 (if)

```
if (x5 == x6) {
    x7 = 1;
} else {
    x7 = 2;
}
x8 = 3;
```

```
       bne x5, x6, else1
iftrue1:
       addi x7, x0, 1
       jal  x0, continue1
else1:
       addi x7, x0, 2
continue1:
       addi x8, x0, 3
```

Ejemplo 2 (do…while)

```
x3 = 0;
x5 = 0;
do {
  x3 = x3 + x5;
  x5++;
} while (x5 < 5);
x12 = x3;
```

```
       add  x3, x0, x0
       add  x5, x0, x0
       addi x6, x0, 5
do1:
       add x3, x3, x5
       addi x5, x5, 1
       blt x5, x6, do1
finwhile:
       add x12, x3, x0
```

Este ejemplo suma los números del 1 al 4 y almacena el resultado final en x12

Ejemplo 3 (while)

```
x3 = 0;
x5 = 0;
while (x5 < 5) {
  x3 = x3 + x5;
  x5++;
}
x12 = x3;
```

```
       add  x3, x0, x0
       add  x5, x0, x0
       addi x6, x0, 5
while1:
       bge x5, x6, finwhile
       add x3, x3, x5
       addi x5, x5, 1
       jal x0, while1
finwhile:
       add x12, x3, x0
```

Este ejemplo suma los números del 1 al 4 y almacena el resultado final en x12

Preguntas:

- Pruebe los programas en el simulador en línea: https://www.cs.cornell.edu/courses/cs3410/2019sp/riscv/interpreter/
- Use alguna IA para aprender que son las instrucciones bne, blt, bge y jal de un microprocesador RISC-V.
- En el ejemplo del if … else:
  - ¿Por qué en el ejemplo del if, si la comparación es de igualdad (`x5 == x6`), el salto se hace con la condición contraria, es decir, con diferente (`bne`)?
  - ¿Por qué se pone una instrucción de salto incondicional `jal` antes de la etiqueta else1?
  - ¿Cómo cambiaría el if para que la condición sea (x5 > x6)?
- En el ejemplo del do…while,
  - ¿Por qué se usa el registro x6 y se inicia con el número 5?
  - ¿Por qué la instrucción `blt` no puede hacer una comparación con el dato inmediato 5?
- En el ejemplo del while,
  - ¿Por qué si la condición del while es (x5 < 5) (menor que) se usa la instrucción de salto `bge` (Branch greater or equal: salto si es mayor o igual?

PARA EL LABORATORIO

A) Sistema mínimo basado en procesador

En un sistema mínimo con procesador, que tenga arquitectura Hardvard, es necesario incluir una memoria de instrucciones (IMEM) y una memoria de datos (DMEM). Se debe recordar que, para acceder a todos los dispositivos, una CPU usa tres buses externos: datos, direcciones y control; y si la arquitectura es Hardvard, estos buses son separados para las instrucciones y los datos como se muestra en la siguiente figura.

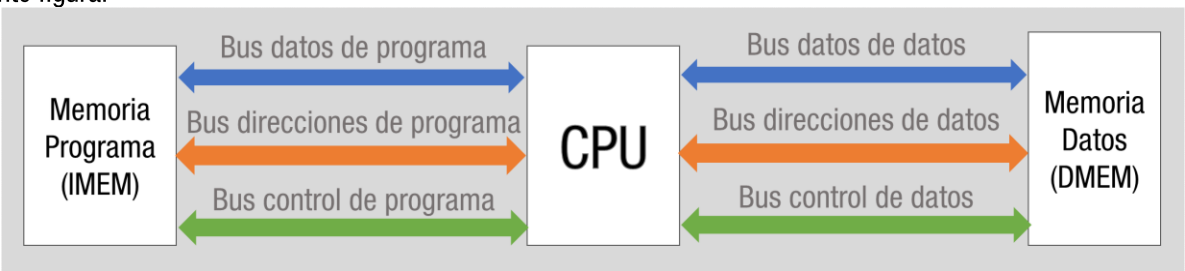

Figura 5. Estructura de un sistema mínimo usando arquitectura Hardvard

Usando un nivel mayor de detalle de un sistema mínimo para un microcontrolador, usaremos la siguiente la interconexión final:

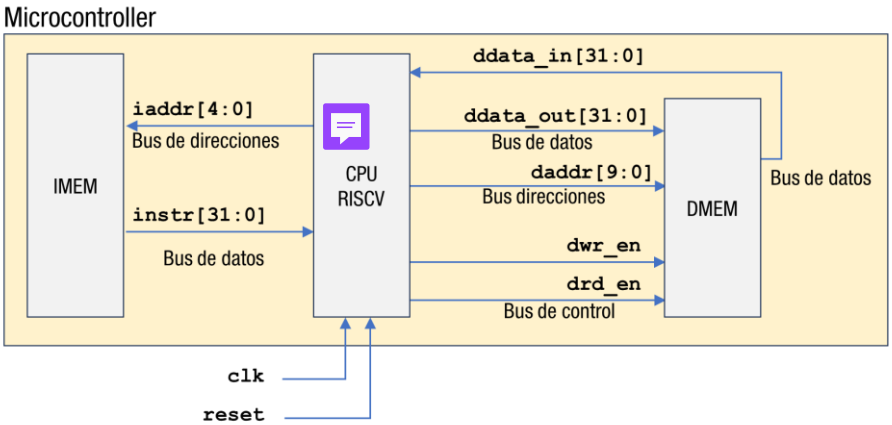

Figura 6. Nuestra implementación para el laboratorio del sistema mínimo basado en procesador RISC-V

Nótese que en la Figura 6, el módulo que se ha venido desarrollando hasta ahora (**riscv**), es un componente del microcontrolador, por lo cual es necesario crear un nuevo módulo que integre nuestra CPU RISC-V con la memoria de programa (IMEM) y la memoria de datos (DMEM). En diseño digital, un módulo que integra otros se conocen como "top module". Asimismo, es necesario crear un nuevo testbench para el microcontrolador. En esta guía implementaremos solamente la memoria de programa. En posteriores guías incluiremos la memoria de datos.

Use la siguiente plantilla para el módulo microcontrolador (en inglés, MCU: MicroController Unit). Note que según la Figura 6, el microcontrolador solamente requiere dos señales de entrada, clk y rst) y tres instancias: una a la CPU RISC-V, otra a la memoria ROM (Read-only memory) para la memoria de programa (IMEM) y otra a la memoria de datos (DMEM).

```
//Definición del microcontrolador
module mcu(
  input clk,  //Reloj del microcontrolador
  input rst   //Reset del microcontrolador
  );

    //Buses con la memoria de programa (IMEM)
    wire [31:0] instr;  //Instrucción de entrada
    wire [31:0] iaddr; //Dirección en la IMEM

    //Buses con la memoria de datos (DMEM)
    wire [31:0] daddr; //Dirección en la DMEM
    wire [31:0] ddata_in;  //Bus de datos DMEM (entrada)
    wire  [31:0] ddata_out; //Bus de datos DMEM (salida)
    wire dwr_en;  //Señales del bus de control DMEM
    wire drd_en;

  // Instancia del microprocesador RISCV
  riscv cpu (
    .rst(rst),
    .clk(clk),
    .instr(instr),
    .iaddr(iaddr),
    .daddr(daddr),
    .ddata_in(ddata_in),
    .ddata_out(ddata_out),
    .dwr_en(dwr_en),
    .drd_en(drd_en)
  );

  // Instancia de la memoria de programa
  rom4096x32 IMEM(
    .data(instr),
    .addr(iaddr[13:2])
  );

endmodule

//Pon aquí la declaración de la memoria ROM de programa
//...
module rom4096x32 (
    input  wire [11:0] addr,      // Dirección (4096 palabras -> 12 bits)
    output reg  [31:0] data       // Datos de 32 bits
);
...
```

Con base en esta plantilla debe rediseñar la especificación de entradas y salidas de su actual módulo **riscv**.

B) Memoria de Instrucciones (IMEM)

En la plantilla del microcontrolador no se ha creado la memoria de programa. Las memorias de programa son generalmente de solo lectura (ROM: Read-only memory) por lo que vamos a crearla e inicializarla con un programa que ejecutará nuestro microprocesador.

Las instrucciones en un microprocesador RISC-V son de 32bits, por lo que éste debe ser el ancho del bus de datos de nuestra IMEM. Además, esta es la razón por la cual se ignoran los bits 0 y 1 de iaddr en la línea 34.

En la IMEM debe estar almacenado un programa en lenguaje de máquina (*machine codes*). En guías pasadas hemos generado el lenguaje de máquina de manera manual, pero es el momento de usar alguna herramienta para automatizar este proceso. Para ello vamos a apoyarnos en una IA para crear una IMEM con un programa inicializado. Dile a la IA: "Crea una memoria ROM con un bus de datos de 32 bits y 4096 palabras en Verilog, que tenga inicializado como datos el siguiente programa en RISC-V RV32I: "…pon aquí el programa del ejemplo 2 (do... while) en ensamblador…".

Revisa muy bien la explicación de la IA de cómo se crea paso a paso la solución e incorpora la IMEM en el módulo top del microcontrolador, pues quizás haya que hacer algunos ajustes en el cableado entre la CPU y la IMEM.

La IA quizás te recomiende que se puede reescribir el código usando el comando $readmemh. Aunque esto lo vamos a hacer más adelante en otra guía, le puedes decir lo que haga y presta mucha atención a las explicaciones que te da. Si quieres puedes usar la implementación que te sugiere usando $readmemh.

chatGPT genera uno machine code incorrecto para la instrucción blt x5, x6, do1, el código correcto debe ser 32'hFE62CCE3, así que corrígelo.

C) Rediseño de la Unidad de Control: Unidad de Fetch

Recuerde que lo que primero que hace la CPU es capturar la instrucción ubicada en la posición apuntada por el PC (Contador de Programa), luego decodificar la instrucción para generar las señales internas y finalmente, ejecutar la instrucción. Estos pasos se conocen como **ciclo de instrucción** y se muestra en la Figura 7.

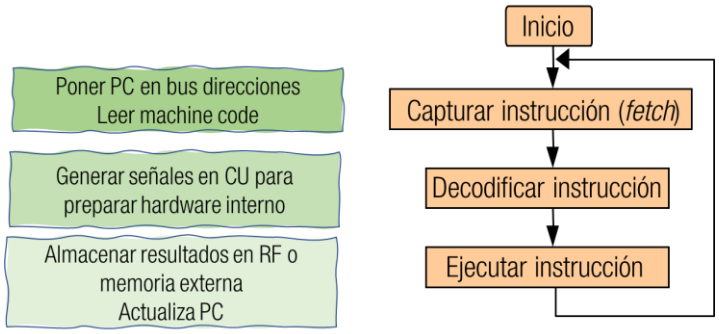

Figura 7. Ciclo de Instrucción

El diseño de la Guía 2 no tenía un estricto control del ciclo de instrucción por lo que se presentaban problemas de almacenamiento en el register file. Lo más simple, para garantizar la adecuada sincronización del sistema, es realizar cada uno de estos pasos en un ciclo de reloj diferente, es decir, el ciclo de instrucción en nuestro diseño tomará 3 ciclos de reloj para completar una instrucción.

Una forma de llevar control del estado dentro del ciclo de instrucción es usar un contador en anillo. Reemplace el código en su implementación donde se declaraba el registro de instrucción IR por:

```
1   reg [31:0] pc;      //Contador de programa
2   assign iaddr = ...;  //Conecta el PC al bus de direcciones de la IMEM
3
4   //Contador en anillo que lleva el registro del estado dentro del ciclo de ejecución
5   reg state_fetch, state_decode, state_execute;
6   always @(negedge clk) begin
7     if (rst) begin
8       state_fetch <= 1;
9       state_decode <= 0;
10      state_execute <= 0;
11    end else begin
12      state_fetch <= state_execute;
13      state_decode <= state_fetch;
14      state_execute <= state_decode;
15    end
16  end
17
18  reg [31:0] IR;  //Registro de instrucción
19  always @(posedge clk) begin
20    if (rst)
21      IR <= 32'd0;
22    else if (state_fetch)
23      IR <= instr;
24  end
25
```

Note que hemos añadido un nuevo registro llamado PC, por lo que debe complementar la línea 2.

En las líneas 5-16 se encuentra toda la lógica para generar tres señales llamadas state_fetch, state_decode y state_execute. Solo una de estas señales está activa al tiempo e indican en qué estado del ciclo de instrucción se encuentra nuestra CPU.

Note también que el registro de instrucción IR se ajustó para tener un reset sincrónico y un habilitador para cargar IR solamente en el ciclo de fetch (state_fetch activo). Esto evita que el IR se cargue en cada ciclo de reloj como ocurría en la implementación previa, lo cual es problemático cuando tenemos una CPU en la que PC puede cambiar durante la ejecución de las instrucciones de salto.

D) Rediseño de la Unidad de Control: Unidad de Decodificación

Nuestro propósito es implementar las instrucciones de salto condicional (Branch) y salto incondicional (Jump) que se muestran en la siguiente tabla:

31 – 25	24 – 20	19 – 15	14 – 12	11 – 7	6 – 0	
imm[20|10:1|11|19:12]				rd	1101111	J jal
imm[12|10:5]	rs2	rs1	000	imm[4:1|11]	1100011	B beq
imm[12|10:5]	rs2	rs1	001	imm[4:1|11]	1100011	B bne
imm[12|10:5]	rs2	rs1	100	imm[4:1|11]	1100011	B blt
imm[12|10:5]	rs2	rs1	101	imm[4:1|11]	1100011	B bge
imm[12|10:5]	rs2	rs1	110	imm[4:1|11]	1100011	B bltu
imm[12|10:5]	rs2	rs1	111	imm[4:1|11]	1100011	B bgeu

Por lo tanto, cree las señales `is_beq, is_bne,` etc., para detectar estas instrucciones. La señal `is_jal` no es necesaria ya que, si mira la guía de instrucciones, `jal` es la única instrucción que usa el formato tipo J, y ya se creó una señal para detectarla: `is_j_instr.`

E) Rediseño de la Unidad de Control: Unidad de Ejecución

En el diseño previo, el valor del registro de destino en el RF tomaba valores extraños antes de terminar cada instrucción. Esto se debe a que en cada ciclo de reloj se podía escribir en el RF. Conforme a lo mostrado en la Figura 7, en el ciclo de instrucción los datos solamente se pueden almacenar en el RF en el ciclo de ejecución (state_execute). Por esta razón debe condicionar el habilitador de escritura del RF a que la señal state_execute esté activa:

```
RegisterFile rv_rf (
    .clk(clk),
    .we(wr_en & state_execute),  //Solo escribe en estado de ejecución
    .waddr( rd ), //Completar
```

F) Implementación de la lógica del PC

El PC es el registro que indica la dirección de memoria desde donde se cargan las instrucciones. Como hemos visto antes, el RV32I usa instrucciones de 32bits por lo que cada instrucción ocupa 4 bytes, lo que significa que el PC debe incrementarse en 4 cada vez que captura y procesa una instrucción. Igualmente, el PC debe iniciarse en 0 después del reset.

Aunque podríamos implementar el PC de manera muy simple creando un contador de 32 bits, es insuficiente, ya que las instrucciones tipo B (Branch) y J (Jump) modifican el valor del PC.

En el programa ejemplo, la instrucción de salto BLT se usa para repetir un bucle interno que acumula los valores hasta el 4. BLT, como todas las instrucciones tipo Branch, es una instrucción de salto condicional que modifica el valor de PC si la condición de prueba es verdadera. Si es falsa, el PC se incrementa en 4 (PC+4). La condición se genera comparando el valor de dos registros fuentes del RF. La implementación de estas instrucciones implica:

- Determinar si el salto se debe tomar (señal `taken_br`)
- Calcular el valor del PC en caso de que el salto se produzca (señal `br_tgt_pc`). Este salto se calcula como PC + Imm. Como Imm es un número con signo, el salto se hace de manera relativa al PC.
- Actualizar el valor del PC según el resultado de la condición o si la instrucción es un salto incondicional (JAL).

Estos pasos implican un diseño del PC como se muestra en la siguiente figura, donde los bloques en verde son los componentes nuevos que debemos agregar.

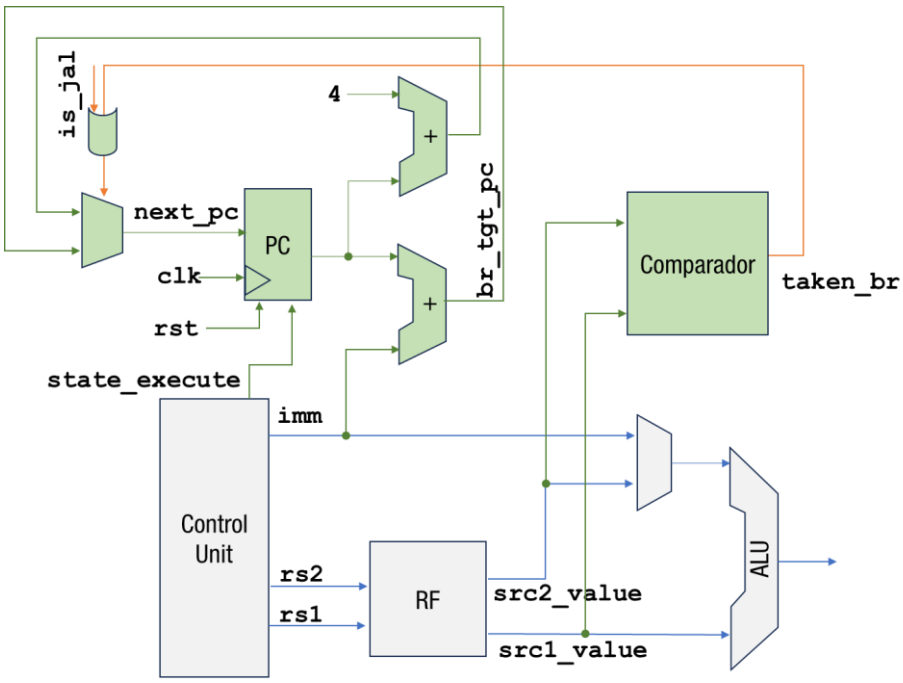

Figura 8 Lógica del Salto Condicional en Detalle

Comencemos con el comparador que genera la condición del salto(`taken_br`). Esta señal se genera según el tipo de instrucción de salto (`is_bne`, `is_blt`, etc.) y los valores almacenados en los dos registros fuente (`src1_value` y `src2_value`, representados como x1 y x2 en la tabla de abajo):

Tabla 1: Condiciones que se dan para cada tipo de instrucción de salto condicional

Instrucción	Significado	Expresión de la condición
BEQ	Branch if equal	`x1 == x2`
BNE	Branch if not equal	`x1 != x2`
BLT	Branch if less than (signed)	`(x1 < x2) ^ (x1[31] != x2[31])`
BGE	Branch if greater than or equal (signed)	`(x1 >= x2) ^ (x1[31] != x2[31])`
BLTU	Branch if less than, unsigned	`x1 < x2`
BGEU	Branch if greater than or equal, unsigned	`x1 >= x2`

Similar a como se implementó la ALU, es posible generar la señal `taken_br` usando el siguiente diagrama:

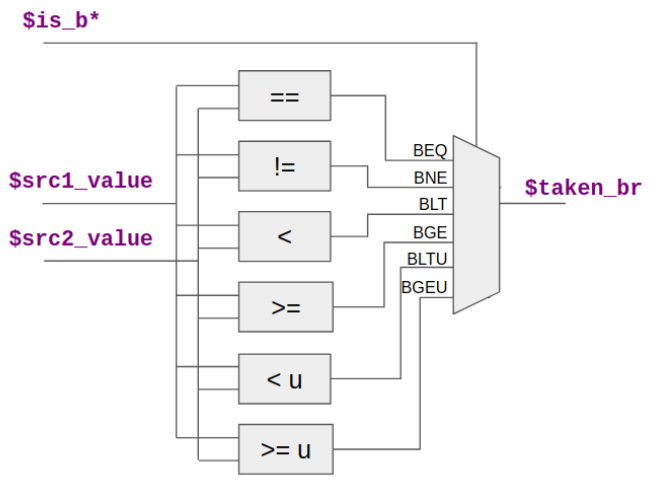

Figura 9 Lógica de la señal taken_br

Misión:

- Codifique la señal `taken_br` usando operadores ternarios para implementar el diagrama de la Figura 9.
- Implemente los dos sumadores de la Figura 8 para generar las señales `br_tgt_pc` y `pc+4`. La señal `br_tgt_pc[31:0]` es la dirección de salto si la condición se cumple en las instrucciones tipo B, o también toma este valor si la instrucción es un salto incondicional JAL.
- Luego implemente el MUX que genera la señal `next_pc`. Note que si la señal `taken_br==1`, el MUX debe entregar `br_tgt_pc`; si es un salto incondicional (instrucción JAL), también debe entregar `br_tgt_pc` y en caso contrario debe ser PC+4.
- Finalmente agregue el siguiente código que genera el registro del PC. Note que el PC es un registro con reset sincrónico y habilitador de carga, donde su valor toma el resultado de la señal next_pc solo en el ciclo de ejecución de la instrucción:

```
always @(posedge clk)
begin
  if (rst)
    pc <= 32'd0;
  else if (state_execute)
    pc <= next_pc;
end
```

El testbench de las guías anteriores ya no nos sirve, porque debemos probar el microcontrolador no solo la CPU RISC-V. Note que el testbench a usar solamente genera la señal de reloj y el reset. Todas las señales internas del microcontrolador las gestionará internamente la CPU RISC-V con base en el programa almacenado en la memoria de programa y siguiendo el ciclo de instrucción.

```
1  `timescale 1ns/1ps
2
3  //Test bench para el microcontrolador
4
5  module tb_mcu;
6
7    // Señales de prueba
8    reg  clk;
9    reg  rst;
10
11   // Instancia del microcontrolador
12   mcu mcuuq (
13     .rst(rst),
14     .clk(clk)
15   );
16
17   integer i;
18   // Bloque inicial: aplica estímulos
19   initial begin
20     // Configurar la generación de archivos de volcado
21     $dumpfile("tb_mcu.vcd");
22     $dumpvars(0, tb_mcu);
23     for(i=0; i<32; i=i+1) $dumpvars(0, mcuuq.cpu.rv_rf.mem[i]);
24
25     #0;  clk = 0;
26      rst = 1;
27
28     #5;  rst = 0;
29
30     #250;  //Termina después de 25 ciclos de reloj
31
32   #16;
33     // Fin de la simulación
34     $finish;
35   end
36
37   always
38     begin
39         #2 clk=!clk;
40     end
41
42 endmodule
```

Simule el programa usando este testbench. Si todo es correcto, el programa debe sumar los números del 1 al 4 en el registro x3, y dejar el resultado final de 10 en x12.

Para analizar el comportamiento del sistema en GTKWave, añada las señales mostradas en la siguiente figura. Debe explorar los diferentes módulos porque algunas de estas señales son de la ALU o el RF.

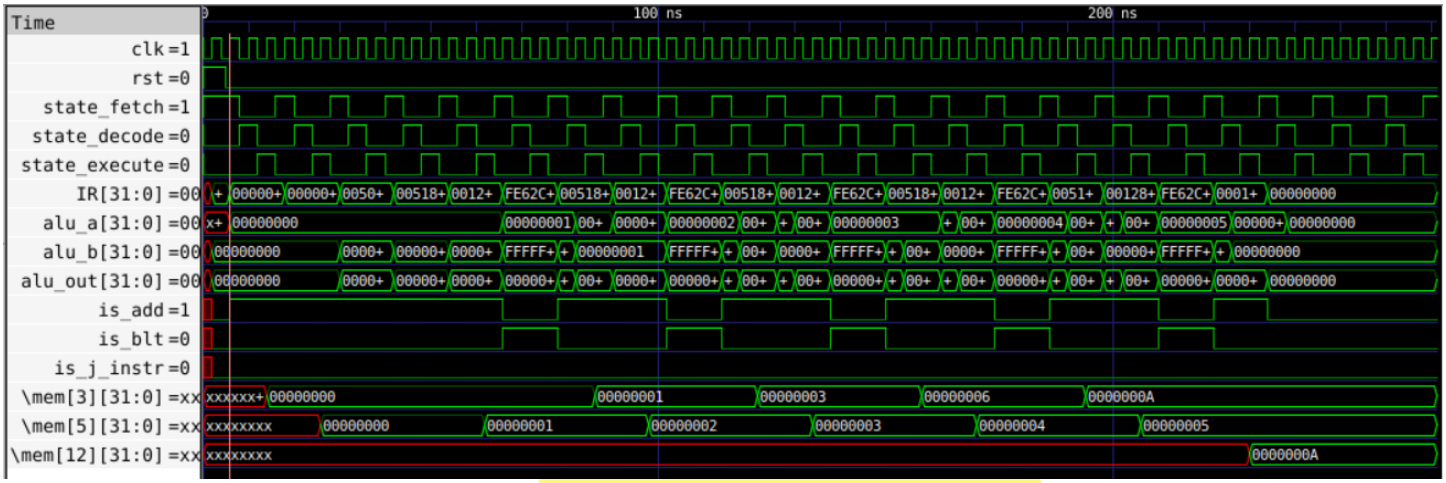

Figura 10. Diagrama de tiempos de la solución final

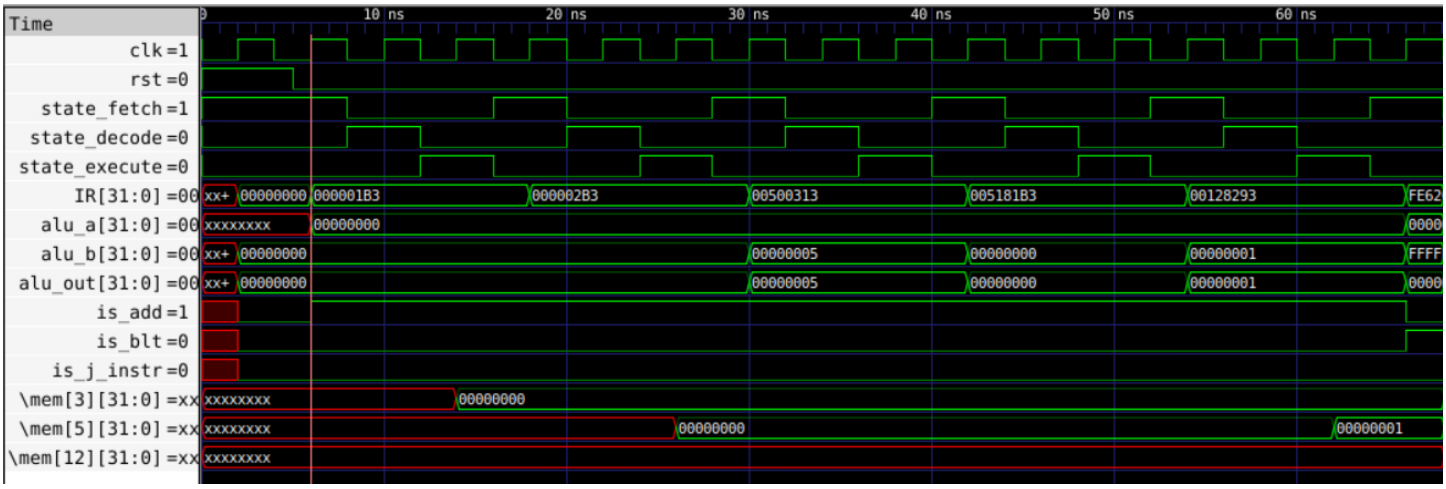

Figura 11. Zoom al diagrama de tiempos donde se observa que en el flanco de subida resaltado se hizo el primer fetch y a partir de allí, en los siguientes flancos de subida, se decodifica y ejecuta la instrucción.

Si llega hasta este punto, ¡felicitaciones ha hecho una primera implementación funcional de un sistema mínimo basado en microprocesador RISC-V!